

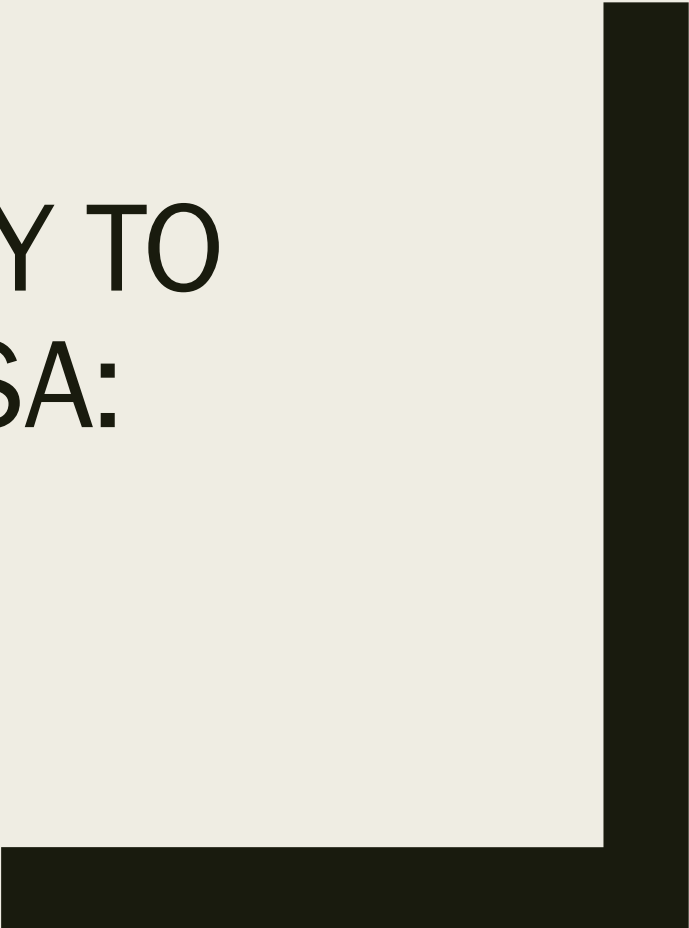


# EXTENDING SOCIOTY TO THRESHOLD ECDSA:

EE-59903 Advanced Cybersecurity

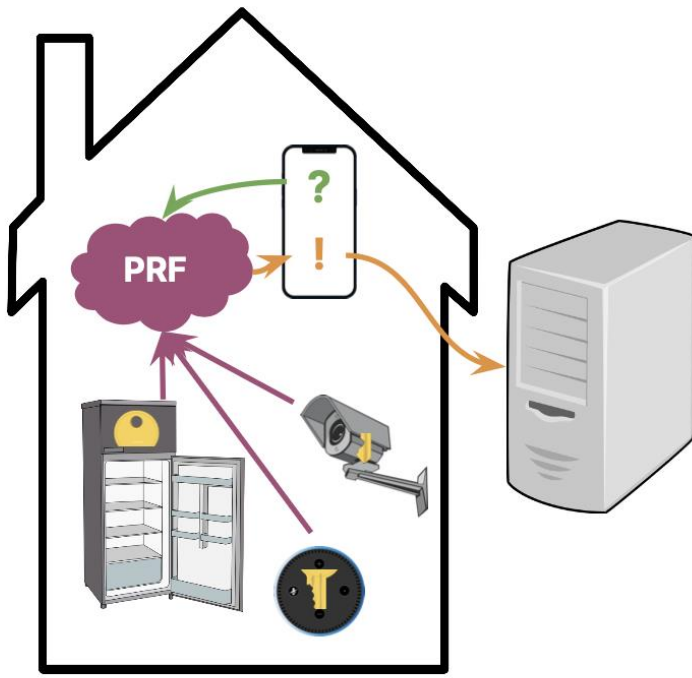
Ma7 19 2026

Angel Coronel



# What is SocloTy?

- Cryptography using smart-home devices – “at-home cryptography”
- Combines smartphone w/ IoT devices
  - $(TV1 + TV2 + Fridge + Thermostat + LightSwitch1 + LightSwitch2 + \dots) + Phone = \text{crypto key or crypto signature}$
- Use distributed PRFs to make a lightweight primitive
- Want to make crypto operations work *only* within the context of a location



**Figure 1: An overview of SocIoTy, which uses a PRF built from IoT devices to provide at-home cryptographic services.**

**Table 1: Hardware specifications of test devices as well as examples of comparable IoT devices.**

| Test Device | CPU            | RAM     | Comparable IoT Device          |
|-------------|----------------|---------|--------------------------------|
| RPi 3B+     | ARM Cortex-A53 | 1 GiB   | Apple TV HD [7]                |
| RPi 2B      | ARM Cortex-A7  | 1 GiB   | Amazon Echo Dot (3rd Gen) [12] |
| RPi Zero W  | ARM1176JZF-S   | 512 MiB | Google Nest Thermostat E [42]  |
| ESP32       | Xtensa LX6     | 320 KiB | Belkin WeMo Light Switch [64]  |

# What is threshold ECDSA?

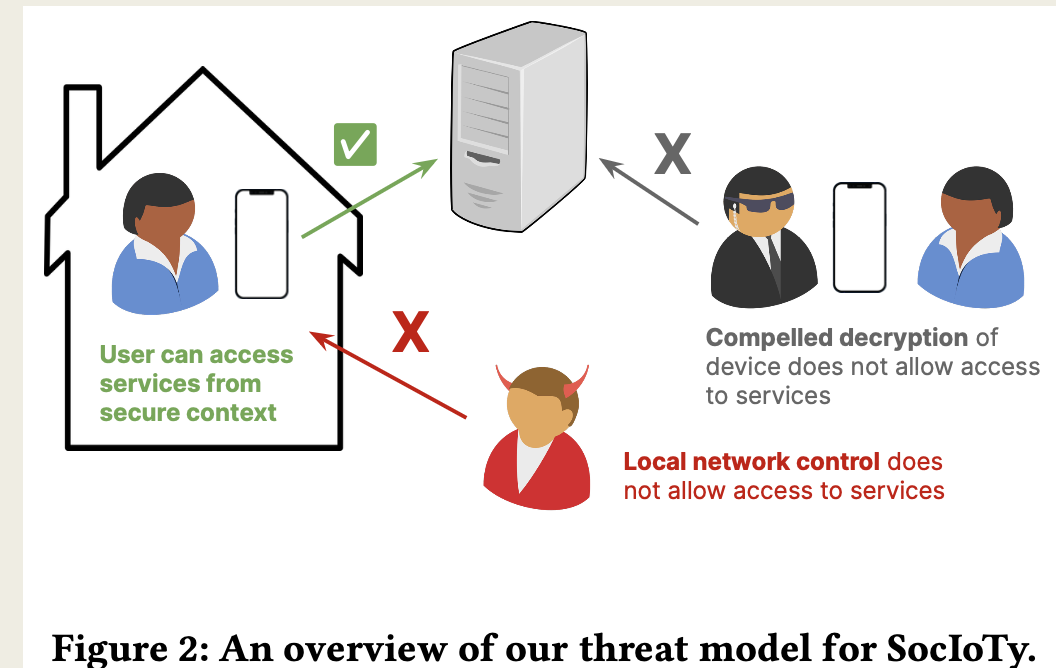
- Elliptic Curve Digital Signature Algorithm
  - *Crypto protocol that allows  $n$  parties to collectively sign messages without any single individual party knowing the full private key (only their share of it  $\sim 1/n$ )*
- Normal ECDSA:
  - *one private key*
  - *one signer*
- Threshold ECDSA:
  - *private key is split into shares*
  - *no single party has the full key*
  - *enough parties cooperate to sign*

# Important terms

- $n$  = total parties/devices
- $t$  = threshold parameter ( $t + 1$  parties needed for signing)
- DKG/keygen = creates distributed key shares
- DSG/signing = produces a signature using a quorum
- PRF
  - an efficient, deterministic algorithm that maps inputs to outputs in a way that appears completely random to any observer who does not know the secret key

# Why is this relevant?

- Smartphones can be hacked or leak important information
  - *They can be lost, stolen, seized, or extracted*
- SocloTy aims to use fixed devices to help protect secrets
- IoT devices are not safe enough so smartphone is needed for access



# My work: can SocloTy support threshold ECDSA signing?

- Originally SocloTy does lightweight cryptography using PRFs
- My project adds & tests threshold ECDSA signing
- Why ECDSA matters:
  - *Web3 wallets*
  - *authentication*
  - *key custody*
  - *compatibility with existing systems*

# Protocols Tested

- Multi-Party Threshold Signature Scheme
  - *based on Gennaro and Goldfeder and implemented by Binance*
  - Used for crypto-currencies such as Bitcoin, Ethereum, NEO and others
  - <https://github.com/bnb-chain/tss-lib>
- DKLS23
  - *Based on Jack Doerner, Yashvanth Kondi, Eysa Lee\*, abhi shelat implemented by Silence Laboratories*
  - <https://github.com/silence-laboratories/dkls23>

| Implementation               | Language | Why tested                                                              |
|------------------------------|----------|-------------------------------------------------------------------------|
| DKLS23 / <code>dkls23</code> | Rust     | More systems-oriented, metrics support, relay model                     |
| Binance <code>tss-lib</code> | Go       | Widely known threshold ECDSA implementation based on GG-style protocols |

# My contributions

- Multi-Party Threshold Signature Scheme
- Ran both protocols locally
- Created benchmark workflows for:
  - *keygen / DKG*
  - *signing / DSG*
  - *repeated signing / multisign*
- Built a DKLS cloud VM orchestration setup using JetStream
  - *Simulated protocol running with 11 VMs*
  - *Built TCP network layer for VM communication with protocol*
- Collected metrics:
  - *runtime*
  - *memory / RSS*
  - *messages bytes*
  - *logs*
  - *per-party artifacts*
- Organized results into CSVs and graphs

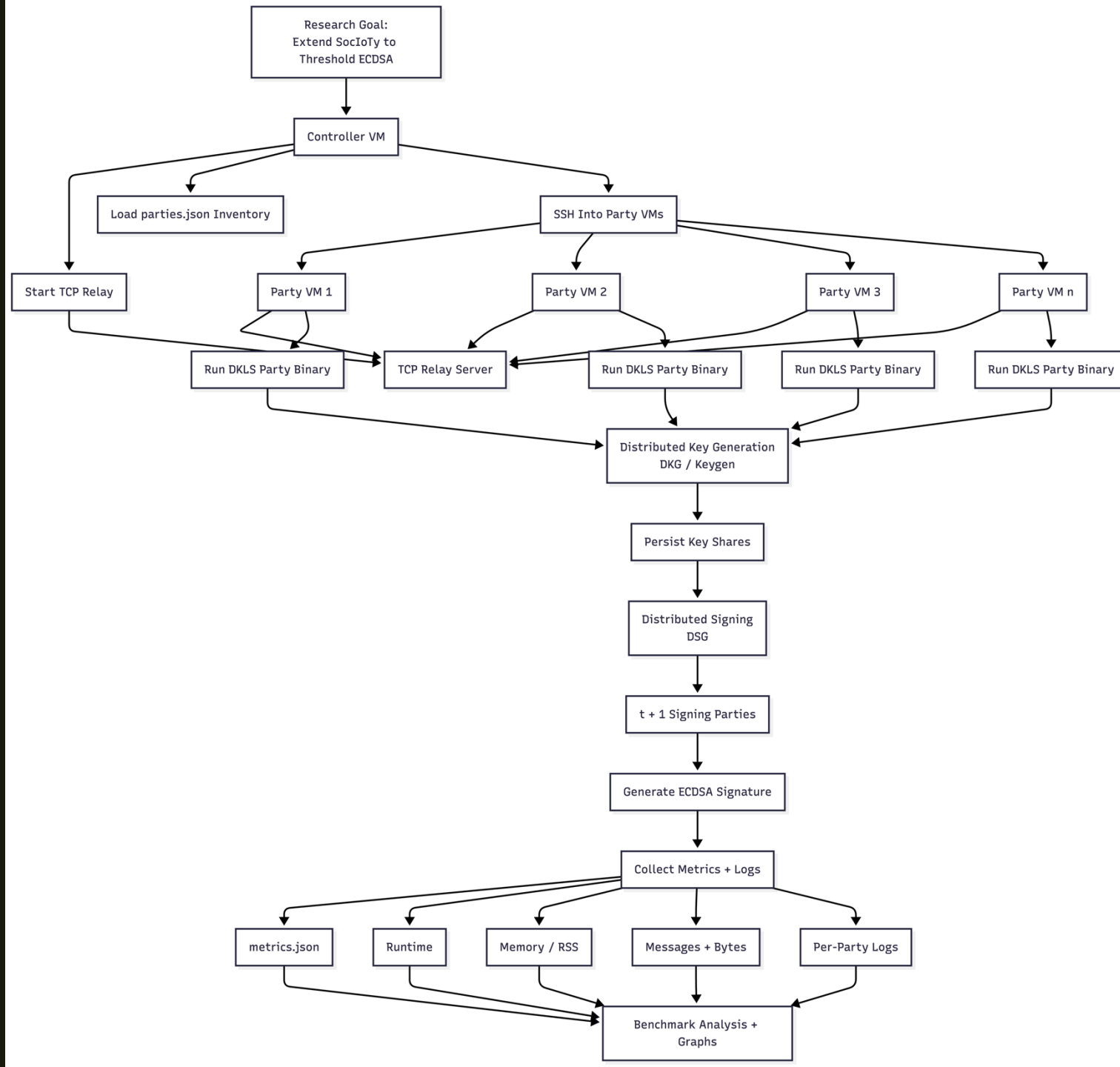
# Plan

## Original

- Run locally & benchmark
  - *Proof of concept*
- Run on protocol cloud
  - *Simulates real environment*
- Run on real, physical micro-controllers & IoT devices
  - *Acts as the real environment*

## Actual

- Ran locally & benchmarked
  - *DKLs & Binance*
- Cloud/JetStream VMs
  - *DKLs completed*
  - *Binance missing*
- IoT devices left for future work
  - *Raspberry Pis + ESP32s purchased but not set-up/tested*



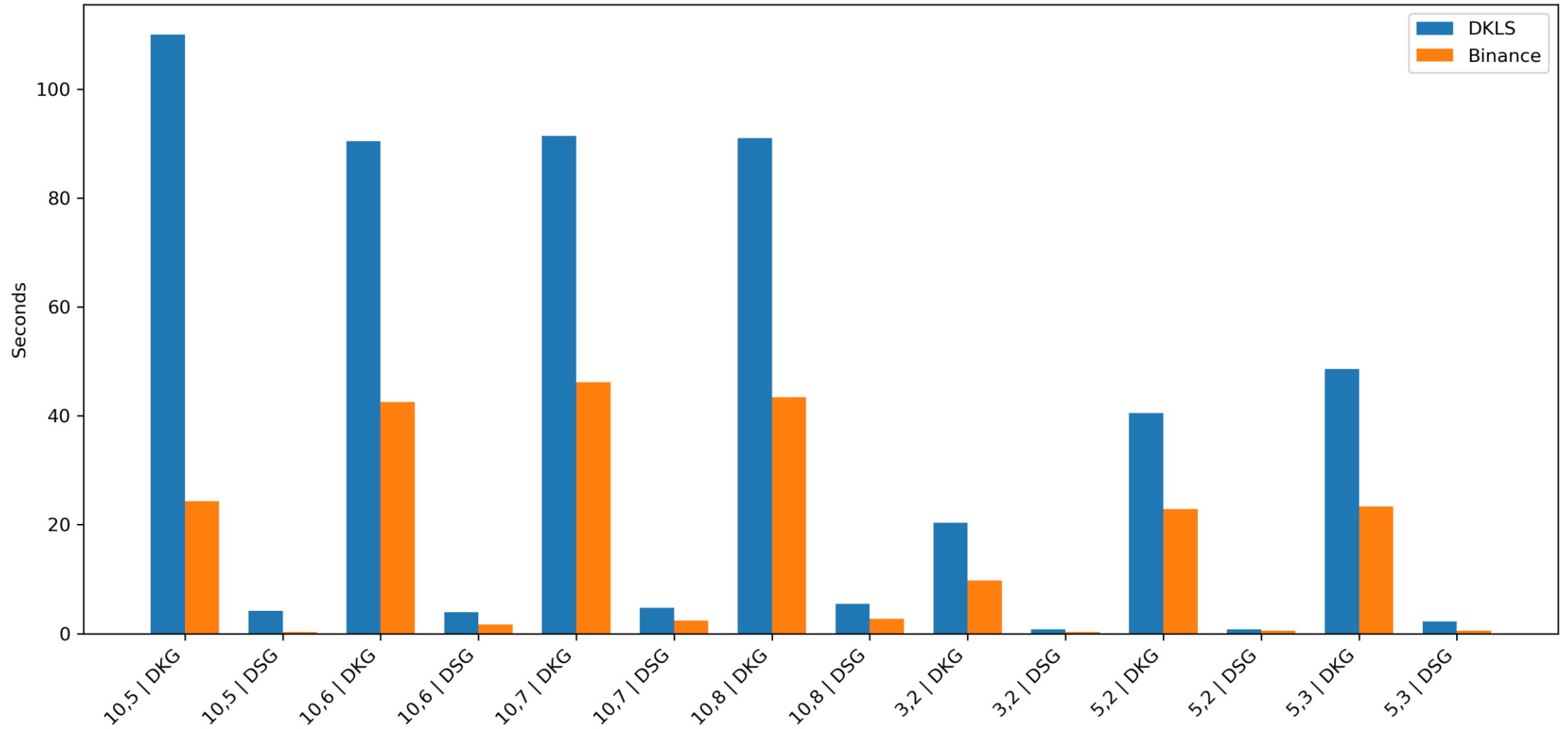
# DKLS cloud architecture

- Made with <https://mermaid.live/>

# Combinations Tested

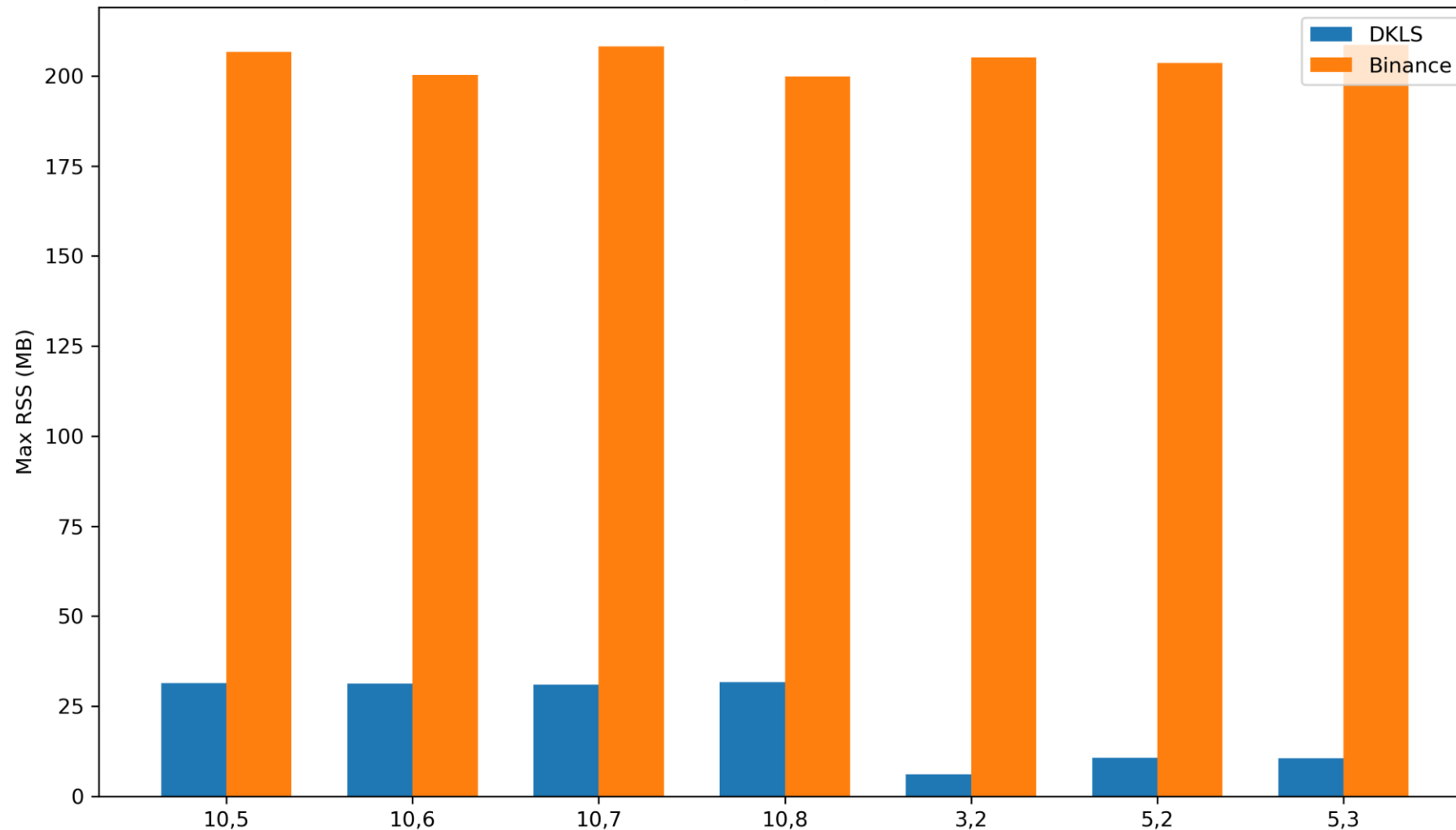
- Tested DKS and DKG on protocols with  $(n, t)$  combinations
  - *both locally*
  - *DKLs on cloud*
- $(3,2)$
- $(5,2)$
- $(5,3)$
- $(10,5)$
- $(10,6)$
- $(10,7)$
- $(10,8)$
- Tested multisign on protocols with  $(n, t)$  combinations then  $x$  amount of repeated signatures on the same key
  - *both locally*
  - *DKLs on cloud*
- $(3,2)$ , signatures: 1, 5, 10, fixed/random
- $(5,3)$ , signatures: 1, 5, 10, fixed/random
- **Fixed mode**
  - *same  $t+1$  signers each round*
- **Random mode**
  - *signer subset changes each round*

Runtime Comparison



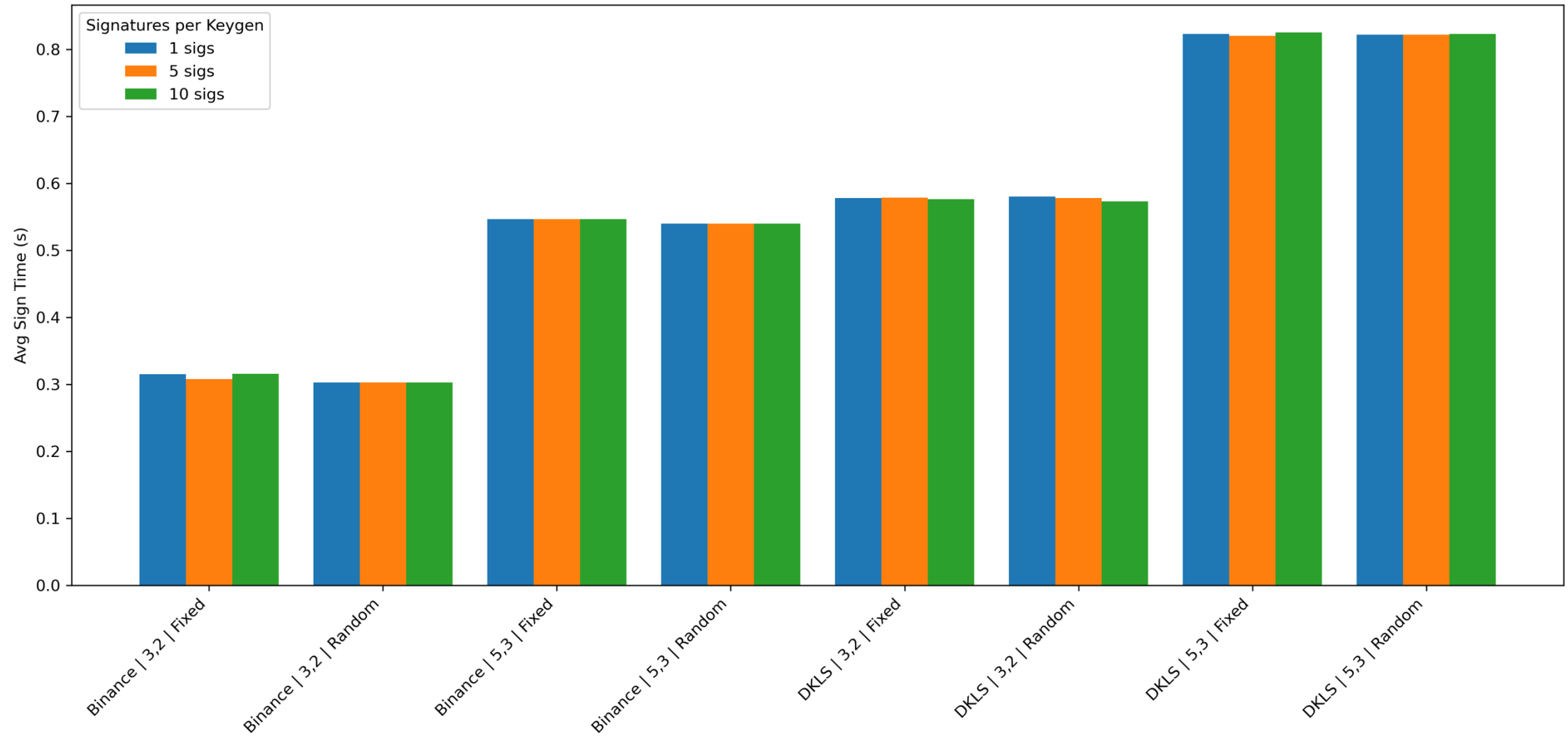
- DKLS has a higher runtime than Binance

Memory Comparison



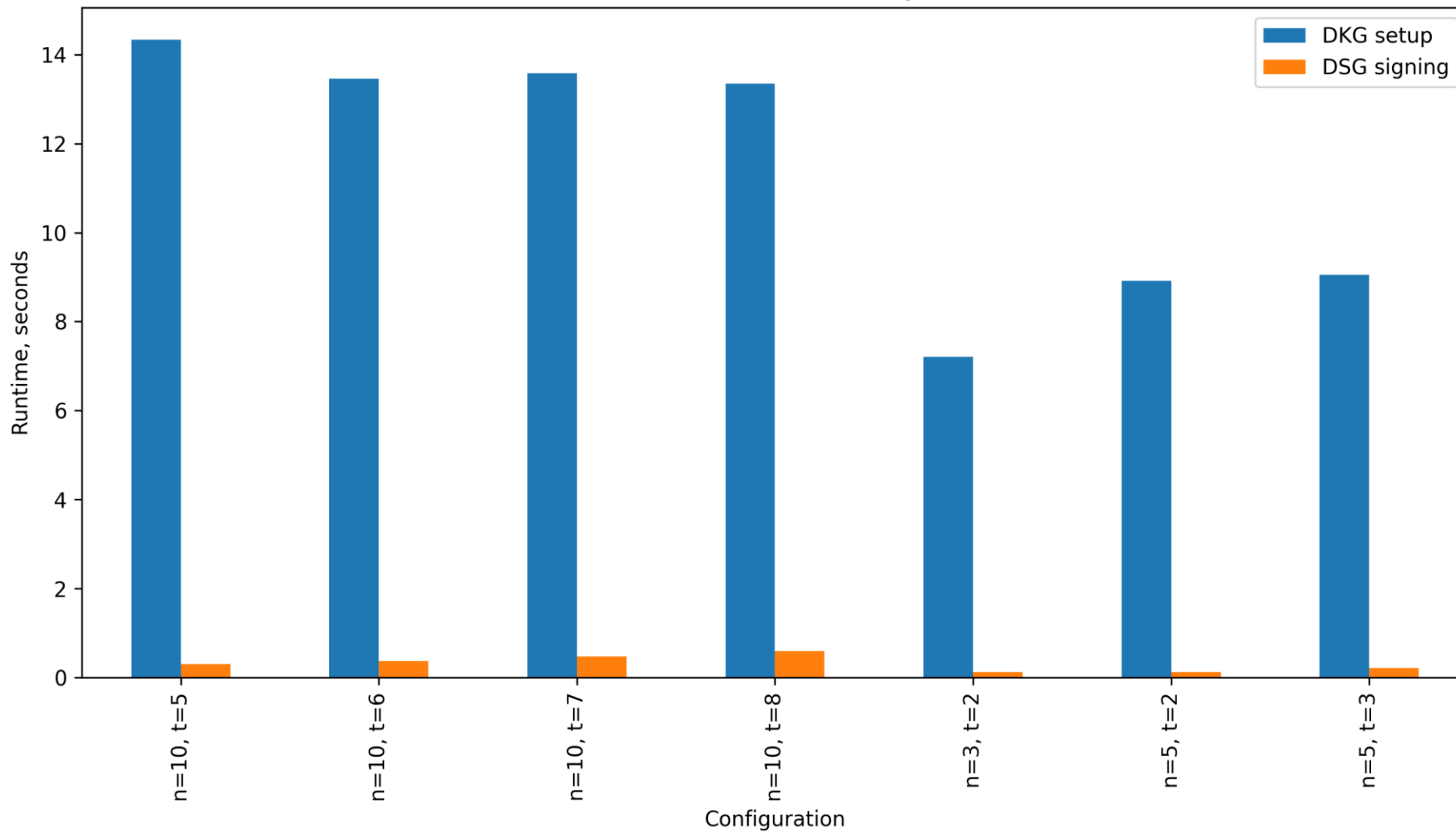
- In comparison, binance has a higher memory consumption

Multi-Sign Comparison - Grouped Bar Chart



- Multiple signatures on same key generation remains relatively the same
- Binance continues to have the best run time overall

DKLS DKG vs DSG Runtime on Jetstream



- On JetStream, setup/keygen takes about 14 seconds for 10 devices and 8 seconds for 5 devices
- KeyGen compared to Signing remains consistent with local tests

# Results/Findings

- DKG/keygen is much more expensive than signing
  - *Signing is the cheaper recurring operation*
- Best to have one keygen done then many signatures used
  - *Average signing cost stays relatively stable*

# DKLs vs Binance

## ■ DKLs:

- *Easier path to real distributed cloud orchestration*
- *Better fit for VM relay experiment*
- *Metrics and relay model helped*

## ■ Binance tss-lib:

- *Good local benchmark harness*
- *Useful comparison point*
- *More system work needed for true multi-VM networking*
- *Local-channel simulation is not the same as real distributed VM execution*

# Limitations

- Don't have real IoT devices being tested as part of results
- Binance testing is missing from JetStream/Cloud tests
  - *Still need a network layer built (maybe a wrapper) for tests*
- Cloud VMs are not great side-by-side comparisons to a real IoT device or microcontroller
- VMs and local machine do not show power consumption in running protocol

# Future Work

- Finish Binance cloud work with multiple VMs
- Run both protocols on Raspberry Pi, Raspberry Pi Zero and ESP32s
- Test if ESP32 are truly feasible
- Add power consumption measurements to project
- Study device churn – devices being taken in or out of the protocol pool
  - *Got rid of old TV and replaced it with new one? That's an example of churn.*

# Citations

- Rosario Gennaro and Steven Goldfeder. Fast Multiparty Threshold ECDSA with Fast Trustless Setup. Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS), 2018.
- Niv Gilboa. Two Party RSA Key Generation. Annual International Cryptology Conference (CRYPTO), LNCS 1666, pp. 116–129, Springer, 1999.
- Tushar M. Jois, Gabrielle Beck, Sofia Belikovetsky, Joseph Carrigan, Alishah Chator, Logan Kostick, Maximilian Zinkus, Gabriel Kaptchuk, and Aviel D. Rubin. SocloTy: Practical Cryptography in Smart Home Contexts. Proceedings on Privacy Enhancing Technologies, 2024(1):447–464, 2024. DOI: 10.56553/popets-2024-0026.
- Carsten Baum, Samuel Dittmer, Peter Scholl, and Xiao Wang. SoK: Vector OLE-Based Zero-Knowledge Protocols. 2023.
- Jetstream2 Getting Started Documentation. Jetstream2 Documentation, 2025. Available online at the Jetstream2 documentation site.
- Silence Laboratories. DKLS23: High-Performance Threshold ECDSA Signing Protocol. GitHub Repository, 2024.
- BNB Chain. Multi-Party Threshold Signature Scheme (tss-lib). GitHub Repository, 2024.